

# Algoritmos e Programação I

Prof. Dr. Amaury Antônio de  
Castro Junior

# Módulo 5 - Estratégias de apoio à programação

Unidade 1 - Documentação e depuração de código

# O que já estudamos?

- Tipos simples (números, caracteres) e comandos básicos de manipulação de dados em Python;
- Estruturas de controle de fluxo (condicionais e laços);
- *Strings* e tipos de dados compostos (vetores e matrizes);
- Fundamentos e benefícios da modularização.

# Depuração e documentação

- Agora que você já domina os fundamentos da programação, entraremos em uma fase essencial para a criação de *softwares* profissionais:
  - a garantia de qualidade e manutenção através da depuração e da documentação.

# Depuração e documentação

- **Depuração** (*debugging*) é o processo sistemático de identificar, analisar e corrigir erros que podem variar de falhas de sintaxe a problemas lógicos complexos.
- **Documentação** divide-se em duas categorias principais: os comentários, que explicam o funcionamento interno ("como"), e as *docstrings*, que funcionam como um manual de uso para módulos e funções ("o quê").

# Por que documentar códigos?

- Clareza para você e para outros desenvolvedores;
- Facilita manutenção e colaboração;
- Ajuda a criar documentação automática (*docstrings*).

# Por que documentar códigos?

- O que são **Docstrings**:
  - Documentação interna de funções e módulos;
  - Ficam logo após a definição (primeira *string* literal);
  - Há uma diferença entre os comentários comuns e o uso de *docstrings*;
  - Sintaxe básica: três aspas duplas ou simples, abrindo e fechando os comentários ("""...""" ou "...").
  - Local: imediatamente após o cabeçalho de definição da função.

# Como criar uma *Docstring*?

- 3 passos:

1. Logo após a definição da função: depois de usar o comando `def` seguido do nome da função, a primeira coisa que você faz é abrir as aspas;
2. Use três Aspas (duplas ou simples). O mais comum é usar aspas duplas.



# Como criar uma *Docstring*?

3. Escreva a documentação: dentro das aspas, você descreve a sua função. É importante incluir:
- **O que a função faz:** um breve resumo do propósito.
  - **Valor(es) de entrada (parâmetros):** quais são os parâmetros, o que cada um significa, qual o tipo esperado (ex: ``float``, ``int``);
  - **Resultado (retorno):** o que a função devolve, qual o tipo desse retorno.

# Exemplo prático de uso da docstring

- Imagine uma função **potencia** que recebe um valor para os valores da base e do expoente e retorna o resultado;
- Sem *docstrings*, temos:

```
def potencia(numero, expoente):  
    return numero ** expoente
```

# Exemplo prático de uso da *docstring*

- Com *docstrings*, temos:

```
def potencia(numero, expoente):  
    """  
    Função que calcula a potência de um número.  
    Valor de entrada:  
    - numero (float): Número a ser calculado e elevado à potência.  
    - expoente (int): Expoente a ser utilizado no cálculo.  
    Resultado:  
    - Retorna o resultado do cálculo de potência (float).  
    """  
    return numero ** expoente
```

# Boas práticas e erros comuns

- Seja conciso no resumo; detalhe apenas o necessário;
- Use linguagem clara e consistente;
- Evite *docstrings* desatualizadas ou vazias;
- Não duplique informação que o código já expressa.

# Vantagens do uso de *docstrings*

- As *docstrings* são uma ferramenta para tornar seu código Python mais organizado, compreensível e profissional;
- Não importa se você está começando agora ou se já é um programador experiente, documentar suas funções é uma prática que só traz benefícios;
- Algumas IDEs incorporam o texto entre aspas na interface, facilitando o trabalho do programador.

# Pra que serve um depurador, afinal?

- **Objetivo:** encontrar e corrigir bugs de lógica que fazem o programa devolver resultados incorretos;
- **Quando usar:** o programa roda (sem erro de sintaxe), mas o comportamento ou o resultado é inesperado;
- **Como o depurador ajuda:** permite executar o código passo a passo e inspecionar os valores das variáveis em cada etapa.

# Pra que serve um depurador, afinal?

- **Vantagem:**
  - Localiza exatamente onde a lógica falha, sem precisar espalhar `print()` pelo código;
- **Termos e explicação:**
  - "*debugger*" (inglês) = depurador (pt);
  - Podemos pensar nele como um "faxineiro" do código que elimina os *bugs* (erros de programação).

# Usando o depurador no Thonny

- A maioria dos ambientes integrados de desenvolvimento (IDEs), como o Thonny já vêm com um depurador embutido;
- Como achar:
  - Geralmente, você vai encontrar uma opção ou um botão chamado "*debug*" na barra de ferramentas;
  - Ao clicar nele, aparece uma janela ou painel de controle do depurador.



# Exemplo prático de depuração

- Vamos usar o depurador Thonny com um exemplo prático;
- Problema: faça um programa que leia uma sequência de valores numéricos e vá acumulando a soma dos valores. A leitura deve ser encerrada quando o usuário digitar o valor -1. Por fim, seu programa deve imprimir o total da soma dos números informados pelo usuário.

# Exemplo prático de depuração

- Solução em Python (vamos usar o depurador):

```
print("Digite uma sequência de números encerrada por -1.")
soma = 0
num = 0
while num != -1:
    num = int(input("Digite um número (-1 encerrar): "))
    soma = soma + num

print("A soma dos números da sequência é: ", soma)
```

# Vantagens e recomendações

- *Docstrings* tornam o código mais utilizável e sustentável;
- Escolha um estilo e mantenha consistência;
- Pratique escrevendo *docstrings* desde o início do projeto.

# Dicas e próximos passos

- Pratique a documentação e a depuração de código com exemplos e exercícios de aulas anteriores;
- Aprender algoritmos exige empenho, dedicação e muitos... muitos exercícios;
- Leia os materiais indicados como referências;
- Busque outras fontes;
- Faça pesquisas e exercícios;
- Comprometa-se com o curso e com a disciplina.

# Referências

BANIN, S. L. **Python 3: conceitos e aplicações - uma abordagem didática**. São Paulo: Erica, 2018. ISBN 9788536530253.

MANZANO, J. A. N. G. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29 ed. São Paulo: Erica, 2019. ISBN 9788536531472.

PERKOVIC, L. **Introdução à computação usando Python: um foco no desenvolvimento de aplicações**. Rio de Janeiro: LTC, 2016. ISBN 9788521630937.

WENTWORTH, P.; ELKNER, J.; DOWNEY, A. B.; MEYERS, C. **How to think like a computer scientist: learning with Python 3**. [S.L.: S. N.], 2012. Disponível em: <https://link.ufms.br/Vow0N>. Acesso em 05 mai. 2025.

# Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

